

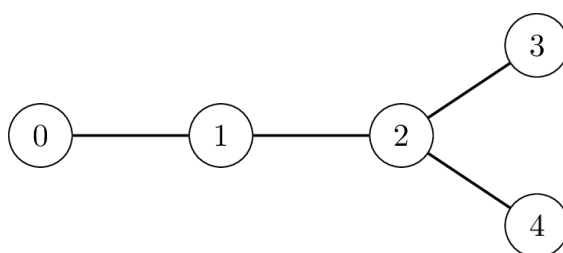
# Migrations

O Museu de História Natural está a investigar os padrões de migração dos dinossauros na Bolívia. Os paleontólogos descobriram pegadas de dinossauros em  $N$  sítios diferentes, numerados de 0 a  $N - 1$  por ordem decrescente de idade: o sítio 0 contém as pegadas mais antigas, enquanto que o sítio  $N - 1$  contém as mais jovens.

Os dinossauros migraram para cada sítio (exceto o sítio 0) a partir de um sítio específico e mais antigo. Para cada sítio  $i$  tal que  $1 \leq i \leq N - 1$ , existe exatamente um sítio mais antigo  $P[i]$  (com  $P[i] < i$ ) tal que alguns dinossauros migraram diretamente do sítio  $P[i]$  para o sítio  $i$ . Um único sítio mais antigo pode ter sido a fonte de migrações para vários sítios mais jovens.

Os paleontólogos modelam cada migração como uma **ligação não dirigida** entre os sítios  $i$  e  $P[i]$ . Nota que, para quaisquer dois sítios distintos  $x$  e  $y$ , é possível viajar de  $x$  para  $y$  percorrendo uma sequência de ligações. A **distância** entre os sítios  $x$  e  $y$  é definida como o número mínimo de ligações necessárias para viajar de  $x$  para  $y$ .

Por exemplo, a imagem a seguir mostra um caso com  $N = 5$  sítios, onde  $P[1] = 0$ ,  $P[2] = 1$ ,  $P[3] = 2$  e  $P[4] = 2$ . É possível viajar do sítio 3 para o sítio 4 através de 2 ligações e por isso a distância entre eles é 2.



O objetivo do Museu é identificar um par de sítios com a maior distância possível.

Nota que o par não é necessariamente único: por exemplo, na figura acima, ambos os pares  $(0, 3)$  e  $(0, 4)$  têm uma distância de 3, que é a máxima. Nestes casos, qualquer dos pares que atinja a distância máxima é considerado válido.

Inicialmente, os valores de  $P[i]$  **não** são conhecidos. O Museu envia uma equipa de investigação para visitar os sítios  $1, 2, \dots, N - 1$ , por ordem. Ao chegar a cada sítio  $i$  ( $1 \leq i \leq N - 1$ ), a equipa executa as duas acções seguintes:

- Determina o valor de  $P[i]$ , ou seja, a origem da migração para o sítio  $i$ .
- Decide se envia **uma** mensagem para o museu ou não envia qualquer mensagem a partir deste sítio, com base nas informações previamente recolhidas.

As mensagens são transmitidas através de um sistema de comunicação por satélite dispendioso, pelo que cada mensagem deve ser um número inteiro entre 1 e 20 000, inclusive. Além disso, a equipa pode enviar **no máximo 50 mensagens** no total.

A tua tarefa é implementar uma estratégia, através da qual:

- A equipa de investigação seleciona os sítios a partir dos quais as mensagens são enviadas, juntamente com o valor de cada mensagem.
- O Museu pode determinar um par de sítios com a distância máxima, com base apenas nas mensagens recebidas de cada sítio, sabendo a partir de que sítios essas mensagens foram enviadas.

O envio de grandes números através do satélite é dispendioso. A tua pontuação dependerá tanto do maior número inteiro enviado, como do número total de mensagens transmitidas.

## Detalhes de Implementação

Tens de implementar duas funções: uma para a equipa de investigação e outra para o Museu.

A função a implementar para a **equipa de investigação** é:

```
int send_message(int N, int i, int Pi)
```

- $N$ : o número de sítios com pegadas.
- $i$ : o índice do sítio que a equipa está atualmente a visitar.
- $Pi$ : o valor de  $P[i]$ .
- Esta função é chamada  $N - 1$  vezes para cada caso de teste, na ordem de  $i = 1, 2, \dots, N - 1$ .

Esta função deve devolver  $S[i]$  especificando a ação tomada pela equipa de investigação no sítio  $i$ :

- $S[i] = 0$ : a equipa de investigação decide não enviar uma mensagem do sítio  $i$ .
- $1 \leq S[i] \leq 20\,000$  a equipa de investigação envia o número inteiro  $S[i]$  como mensagem do sítio  $i$ .

A função a implementar para o **Museu** é:

```
std::pair<int,int> longest_path(std::vector<int> S)
```

- $S$ : um vector de comprimento  $N$  tal que:
  - $S[0] = 0$ .

- Para cada  $1 \leq i \leq N - 1$ ,  $S[i]$  é o número inteiro devolvido por `send_message(N, i, Pi)`.
- Esta função é chamada exatamente uma vez para cada caso de teste.

Esta função deve devolver um par de sítios  $(U, V)$  com a distância máxima.

Na avaliação real, um programa que chama os procedimentos acima é executado **exatamente duas vezes**.

- Durante a primeira execução do programa:
  - `send_message` é chamada exatamente  $N - 1$  vezes.
  - **O teu programa pode armazenar e reter informação ao longo de chamadas sucessivas.**
  - Os valores devolvidos (vector  $S$ ) são armazenados pelo sistema de avaliação.
  - Em alguns casos, o comportamento do avaliador é **adaptativo**. Isto significa que o valor de  $P[i]$  numa chamada para `send_message` pode depender das acções tomadas pela equipa de investigação durante as chamadas anteriores.
- Durante a segunda execução do programa:
  - `longest_path` é chamada exatamente uma vez. A única informação disponível para `longest_path` desde a primeira execução é o vector  $S$ .

## Restrições

- $N = 10\,000$
- $0 \leq P[i] < i$  para cada  $i$  tal que  $1 \leq i \leq N - 1$ .

## Subtarefas

| Subtarefa | Pontuação | Restrições adicionais                                                                  |
|-----------|-----------|----------------------------------------------------------------------------------------|
| 1         | 30        | O sítio 0 e algum outro sítio têm uma distância máxima entre todos os pares de sítios. |
| 2         | 70        | Nenhuma restrição adicional.                                                           |

Seja  $Z$  o número inteiro máximo que aparece no vector  $S$  e seja  $M$  o número de mensagens enviadas pela equipa de investigação.

Em qualquer dos casos de teste, se pelo menos uma das seguintes condições se verificar, então a pontuação da tua solução para esse caso de teste será 0 (reportada como `Output isn't correct` no CMS):

- Pelo menos um elemento em  $S$  é inválido.
- $Z > 20\,000$  ou  $M > 50$ .
- O valor devolvido pela chamada à função `longest_path` está incorreto.

Caso contrário, a tua pontuação na subtarefa 1 é calculada da seguinte forma:

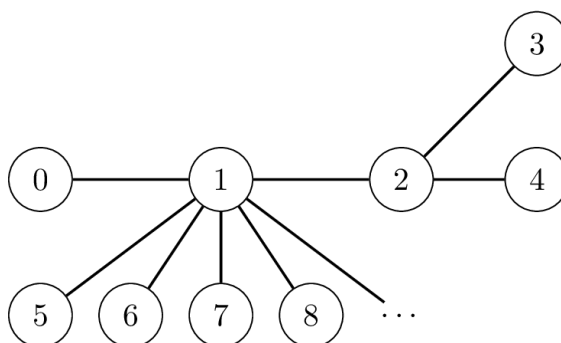
| Condição                     | Pontuação |
|------------------------------|-----------|
| $9\,998 \leq Z \leq 20\,000$ | 10        |
| $102 \leq Z \leq 9997$       | 16        |
| $5 \leq Z \leq 101$          | 23        |
| $Z \leq 4$                   | 30        |

A tua pontuação na subtarefa 2 é calculada da seguinte forma:

| Condição                              | Pontuação                                        |
|---------------------------------------|--------------------------------------------------|
| $5 \leq Z \leq 20\,000$ e $M \leq 50$ | $35 - 25 \log_{4000} \left( \frac{Z}{5} \right)$ |
| $Z \leq 4$ e $32 \leq M \leq 50$      | 40                                               |
| $Z \leq 4$ e $9 \leq M \leq 31$       | $70 - 30 \log_4 \left( \frac{M}{8} \right)$      |
| $Z \leq 4$ e $M \leq 8$               | 70                                               |

## Exemplo

Seja  $N = 10\,000$ . Considera a situação em que  $P[1] = 0$ ,  $P[2] = 1$ ,  $P[3] = 2$ ,  $P[4] = 2$ , e  $P[i] = 1$  para  $i > 4$ .



Supõe que a estratégia da equipa de investigação consiste em enviar a mensagem  $10 \cdot V + U$  sempre que o par de sítios  $(U, V)$  com a distância máxima muda como resultado de uma chamada a `send_message`.

Inicialmente, o par com a distância máxima é  $(U, V) = (0, 0)$ . Considera a seguinte sequência de chamadas durante a primeira execução do programa:

| Chamada de função                      | $(U, V)$ | Valor devolvido ( $S[i]$ ) |
|----------------------------------------|----------|----------------------------|
| <code>send_message(10000, 1, 0)</code> | $(0, 1)$ | 10                         |
| <code>send_message(10000, 2, 1)</code> | $(0, 2)$ | 20                         |
| <code>send_message(10000, 3, 2)</code> | $(0, 3)$ | 30                         |
| <code>send_message(10000, 4, 2)</code> | $(0, 3)$ | 0                          |

Nota que em todas as chamadas restantes o valor de  $P[i]$  é 1. Isto significa que o par com a distância máxima não se altera, e a equipa não envia mais mensagens.

De seguida, na segunda execução do programa, é feita a seguinte chamada:

```
longest_path([0, 10, 20, 30, 0, ...])
```

O Museu lê a última mensagem enviada pela equipa de investigação, que é  $S[3] = 30$ , e deduz que  $(0, 3)$  é o par de sítios com a distância máxima. Desse modo, a chamada devolve  $(0, 3)$ .

Nota que esta abordagem nem sempre permite ao Museu determinar corretamente o par com a distância máxima.

## Avaliador Exemplo

O avaliador exemplo chama ambas as funções `send_message` e `longest_path` na mesma execução, o que é diferente do avaliador real.

Formato do input:

```
N
P[1] P[2] ... P[N-1]
```

Formato do output:

```
S[1] S[2] ... S[N-1]
U V
```

Nota que é possível utilizar o avaliador exemplo com qualquer valor de  $N$ .